

Problem A. Good Contest

时间限制: 1 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

下一场编程竞赛包含三道题目，共有 n 名参赛者。

第 1 题是简单题，第 2 题是中等题，第 3 题是困难题。

如果一名参赛者没有解出全部三道题目，则称该参赛者为“较弱的”(weak)。

不幸的是，记分板丢失了。仅存的信息是一个长度为 3 的数组 a ，其中 a_i 表示解出第 i 题的参赛者人数。

在所有与此信息相符的记分板中，求弱参赛者的最小可能数量。

输入格式

第一行包含一个整数 t ($1 \leq t \leq 3000$) —— 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($1 \leq n \leq 9$) —— 参赛者的人数。

每个测试用例的第二行包含三个整数 a_1, a_2, a_3 ($0 \leq a_i \leq n$)，其中 a_i 表示解出第 i 题的参赛者人数。

输出格式

对于每个测试用例，输出一个整数——弱参赛者的最小可能数量。

样例

标准输入流	标准输出流
6	0
3	1
3 3 3	0
4	1
4 4 3	5
1	4
1 1 1	
9	
9 8 9	
5	
0 5 5	
6	
4 3 2	

提示

在第一个测试用例中，所有 3 名参赛者都可以解出全部三道题，因此答案为 0。

在第二个测试用例中，参赛者 1 可以只解出第 1 题和第 2 题，而参赛者 2、3 和 4 解出了全部三道题。因此，参赛者 1 是唯一的弱参赛者，答案为 1。可以证明这是最小值。

Problem B. Three Piles

时间限制: 1 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

Alice 和 Bob 正在用三堆石子玩一个游戏。最初, Alice 拥有 a 枚石子, Bob 拥有 b 枚石子, 第三堆包含 c 枚石子。

Alice 和 Bob 轮流行动, Alice 先手。在每个回合中, 当前行动的玩家可以从第三堆石子中取走任意数量的石子 (可以是 0 枚), 并将其加入到自己的石子堆中。

如果两位玩家在连续两个回合中均取走了 0 枚石子, 则游戏结束。

设 A 和 B 分别为 Alice 和 Bob 最终拥有的石子数量。游戏的得分为 $|A - B|$ 。

Alice 希望最大化最终得分, 而 Bob 希望最小化最终得分。假设双方均采用最优策略, 求最终的游戏得分。

输入格式

第一行包含一个整数 t ($1 \leq t \leq 10^4$) —— 测试用例的数量。

每个测试用例包含一行, 包含三个整数 a, b, c ($0 \leq a, b, c \leq 10^9$) —— 分别表示 Alice、Bob 初始拥有的石子数量以及第三堆石子的数量。

输出格式

对于每个测试用例, 输出一个整数——双方均采用最优策略时的最终得分。

请注意在 C++ 中使用 64 位整数类型 (例如 `long long`)。

样例

标准输入流	标准输出流
5	3
3 6 3	7
3 6 10	4
5 5 4	3
2 5 6	1024506979
67676767 41414141 998244353	

提示

在第一个测试用例中, Alice 可以在她的第一个回合中选择取走 0 枚石子。随后 Bob 也可以选择取走 0 枚石子, 游戏结束, 两人最终拥有的石子数量分别为 3 和 6。因此, 得分为 3。可以证明 Alice 无法获

得更高的得分，**Bob** 也无法使得分更低。

在第二个测试用例中，**Alice** 可以在她的第一个回合中从第三堆取走全部 10 枚石子。随后游戏结束，两人最终拥有的石子数量分别为 13 和 6，因此得分为 7。

Problem C. AND, OR, Sort!

时间限制: 2 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

给定一个长度为 n 的 01 字符串 s （仅包含字符 0 和 1）。

你可以执行以下操作任意次（也可以是零次）：

- 选择一个整数 i ($1 \leq i \leq n$)，并将 s_i 替换为 $s_1, s_2, \dots, s_i$ 的按位与（bitwise AND）或按位或（bitwise OR）的结果。

注意，单个元素的按位与或按位或等于该元素本身。

你的目标是使 s 按非降序排序（即满足 $s_1 \leq s_2 \leq \dots \leq s_n$ ）。

求将 s 按非降序排序所需的最少操作次数。

输入格式

第一行包含一个整数 t ($1 \leq t \leq 10^4$) —— 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($2 \leq n \leq 2 \cdot 10^5$) —— 01 字符串 s 的长度。

每个测试用例的第二行包含长度为 n 的 01 字符串 s 。 s 中的每个字符要么是 0，要么是 1。

保证所有测试用例的 n 之和不超过 $2 \cdot 10^5$ 。

输出格式

对于每个测试用例，输出一个整数——将 s 按非降序排序所需的最少操作次数。

样例

标准输入流	标准输出流
6	0
4	3
0011	1
4	2
1000	3
5	1
01000	
8	
01001101	
7	
0101010	
7	
0111101	

提示

在第一个测试用例中，字符串已经是有序的，因此不需要任何操作。

在第二个测试用例中，我们可以使用按位或操作将最后三个字符变成 1，在 3 次操作后得到 1111。

Problem D. Falling Concrete

时间限制: 2 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

Vihaan 正在修复一条由 n 个路段组成的道路。第 i 个路段的高度为 a_i 。

他有一辆可以移动路段的叉车。在一次操作中，Vihaan 可以选择两个下标 i 和 j ($1 \leq i < j \leq n$)，铲起第 j 个路段，并将其向后移至位置 i 。

当该路段被向后搬运时，它所经过的每个路段上都会掉落来自该路段的 1 个单位的混凝土。随后，被搬运的路段被插入到位置 i 。

更形式化地，子数组

$$[a_i, a_{i+1}, \dots, a_{j-1}, a_j]$$

被替换为

$$[a_j - (j - i), a_i + 1, a_{i+1} + 1, \dots, a_{j-1} + 1]。$$

如果道路的某个部分是由高度相同的一段连续路段组成的，则称该部分是“平坦的”(flat)。

Vihaan 可以执行任意次操作（也可以是零次）。

求道路中平坦部分的最大可能长度。

输入格式

第一行包含一个整数 t ($1 \leq t \leq 10^4$) —— 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($1 \leq n \leq 2 \cdot 10^5$) —— 道路的路段数。

每个测试用例的第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($n \leq a_i \leq 10^9$) —— 各路段的初始高度。

保证所有测试用例的 n 之和不超过 $2 \cdot 10^5$ 。

可以证明，在给定的限制条件下，经过任意操作序列后，每个路段的高度均保持为正数。

输出格式

对于每个测试用例，输出一个整数——道路中平坦部分的最大可能长度。

样例

标准输入流	标准输出流
8	4
5	6
5 5 5 9 8	1
6	5
6 6 6 6 6 6	4
5	2
5 6 7 8 9	5
5	6
9 7 12 10 12	
4	
4 7 5 8	
7	
14 9 14 12 8 11 12	
5	
1000000000 1000000000 1000000000 1000000000	
1000000000	
8	
8 8 12 8 14 10 15 13	

提示

在第一个测试用例中，Vihaan 可以将位置 4 的路段移动到位置 1:

$$[5, 5, 5, 9, 8] \rightarrow [6, 6, 6, 6, 8]$$

这样就可以创建一个长度为 4 的平坦部分。

在第四个测试用例中，Vihaan 可以执行以下操作:

1. 将位置 3 的路段移动到位置 1:

$$[9, 7, 12, 10, 12] \rightarrow [10, 10, 8, 10, 12]$$

2. 将位置 4 的路段移动到位置 3:

$$[10, 10, 8, 10, 12] \rightarrow [10, 10, 9, 9, 12]$$

3. 将位置 5 的路段移动到位置 3:

$$[10, 10, 9, 9, 12] \rightarrow [10, 10, 10, 10, 10]$$

这样，整条道路都可以变得平坦。

在第五个测试用例中，Vihaan 可以先将位置 2 的路段移动到位置 1:

$$[4, 7, 5, 8] \rightarrow [6, 5, 5, 8]$$

然后将位置 4 的路段移动到位置 2:

$$[6, 5, 5, 8] \rightarrow [6, 6, 6, 6]$$

这样，整条道路也可以变得平坦。

Problem E. Prime Destruction

时间限制: 2 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

给定一个由 n 个正整数组成的多重集合 a 。

你可以执行以下操作任意次（也可以是零次）：

- 从多重集合中选择一个整数 $x > 1$ 以及 x 的一个质因数 p 。从多重集合中移除一个 x ，并向其中加入 p 个 $\frac{x}{p}$ 。

另外还给定一个整数 k ($1 \leq k \leq n$)。设 $f(k)$ 为从原始多重集合开始，使得多重集合中的每个整数均不超过 k 所需的最少操作次数。

求 $f(k)$ 的值。

输入格式

每个测试点包含多组测试用例。第一行包含一个整数 t ($1 \leq t \leq 10^4$) —— 测试用例的数量。接下来是每个测试用例的描述。

每个测试用例的第一行包含两个整数 n 和 k ($1 \leq k \leq n \leq 2 \cdot 10^5$) —— 分别表示多重集合的初始大小以及给定的整数。

每个测试用例的第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) —— 多重集合中的元素。

保证所有测试用例的 n 之和不超过 $2 \cdot 10^5$ 。

输出格式

对于每个测试用例，输出一个整数，表示 $f(k)$ 的值。

样例

标准输入流	标准输出流
6	0
1 1	4
1	25
6 2	15
6 6 4 3 2 1	1
8 1	0
8 6 4 3 2 1 8 6	
12 3	
12 10 9 8 7 6 5 4 3 2 1 12	
10 9	
10 9 8 7 6 5 4 3 2 1	
5 5	
5 4 3 2 1	

提示

在第一个测试用例中，唯一的元素已经不超过 k ，因此不需要进行任何操作。

在第二个测试用例中，我们可以执行以下操作：

- $[6, 6, 4, 3, 2, 1] \rightarrow [2, 2, 2, 6, 4, 3, 2, 1]$
- $[2, 2, 2, 6, 4, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 4, 3, 2, 1]$
- $[2, 2, 2, 2, 2, 2, 4, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 1]$
- $[2, 2, 2, 2, 2, 2, 2, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 2, 1]$

因此，执行 4 次操作即可满足要求。可以证明不存在操作次数严格更少的操作序列。

Problem F. MEX Replacement

时间限制: 2 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

给定一个以压缩形式表示的非负整数多重集合。也就是说，对于多重集合中的每个数值 x_i ，已知其出现次数 y_i 。

在一次操作中，你可以从当前多重集合中任意选择若干个元素（非空子集），将选中的所有元素移除，并向多重集合中加入一个等于它们 MEX^* 的元素。

你可以执行上述操作任意次（也可以是零次）。

求在任意时刻多重集合中可能出现的最大整数。

* 一组整数 $c_1, c_2, \dots, c_k$ 的最小排除数（ MEX , minimum excluded）是指不在该组整数中出现的最小非负整数 x 。

输入格式

第一行包含一个整数 t ($1 \leq t \leq 10^4$) —— 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($1 \leq n \leq 2 \cdot 10^5$) —— 初始多重集合中不同整数的数量。

接下来的 n 行，每行包含两个整数 x_i 和 y_i ($0 \leq x_i \leq 10^9$, $1 \leq y_i \leq 10^9$)，表示整数 x_i 在初始多重集合中恰好出现了 y_i 次。

保证整数 $x_1, x_2, \dots, x_n$ 两两不同。

保证所有测试用例的 n 之和不超过 $2 \cdot 10^5$ 。

输出格式

对于每个测试用例，输出一个整数——在任意时刻多重集合中可能出现的最大整数。

样例

标准输入流	标准输出流
10	5
4	10
2 2	3
0 3	4
4 1	4
1 2	7
1	4
10 1	3
1	59
2 1	5
3	
0 1	
2 2	
1 1	
3	
0 4	
1 2	
2 1	
1	
0 100	
2	
2 7	
3 1	
2	
0 1	
1 1	
3	
0 1	
1 1	
59 1	
4	
0 5	
1 1	
2 3	
3 1	

提示

在第一个测试用例中，初始多重集合为 $\{0, 0, 0, 1, 1, 2, 2, 4\}$ 。

我们可以将 $\{0, 1, 2\}$ 替换为 3，得到 $\{0, 0, 1, 2, 3, 4\}$ 。然后，我们可以将 $\{0, 1, 2, 3, 4\}$ 替换为 5。

可以证明不可能出现大于 5 的整数，因此答案为 5。

在第二个测试用例中，10 已经出现在初始多重集合中，且不可能产生更大的整数，因此答案为 10。

Problem G. Modular Tree

时间限制: 3 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

Vihaan 拥有一棵包含 n 个节点、以节点 1 为根的有根树^{*}。

每个节点 i 都有一个初始权值 a_i 和一个模数 b_i 。设 x_i 表示节点 i 的当前权值。初始时， $x_i = a_i$ 。

Vihaan 可以执行以下操作任意次（也可以是零次）：

- 选择一个节点 u 。令 s 为 u 的所有直接子节点的当前权值之和，然后将 x_u 替换为 $(x_u + s) \bmod b_u$ 。

在执行任意次操作后，Vihaan 希望最大化所有节点的权值之和。

求所有节点权值之和的最大可能值。

^{*} 树是一个无环的无向连通图。

输入格式

第一行包含一个整数 t ($1 \leq t \leq 10^4$) —— 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($1 \leq n \leq 2 \cdot 10^5$) —— 树中的节点数。

每个测试用例的第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i < b_i$) —— 各节点的初始权值。

每个测试用例的第三行包含 n 个整数 $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$) —— 各节点的模数。

接下来的 $n - 1$ 行，每行包含两个整数 u 和 v ($1 \leq u, v \leq n$)，表示节点 u 与节点 v 之间存在一条边。

保证给定的边构成一棵树。

保证所有测试用例的 n 之和不超过 $2 \cdot 10^5$ 。

输出格式

对于每个测试用例，输出一个整数——在执行任意次操作后，所有节点权值之和的最大可能值。

样例

标准输入流	标准输出流
8	3
1	7
3	11
7	6
2	18
0 3	12
5 4	27
1 2	3999999996
3	
0 2 3	
7 3 4	
1 2	
2 3	
3	
0 0 1	
5 2 2	
1 2	
2 3	
4	
1 2 3 4	
10 3 4 5	
1 2	
1 3	
1 4	
3	
0 1 3	
10 2 4	
1 2	
1 3	
5	
0 0 1 2 3	
12 6 9 3 4	
1 2	
1 3	
2 4	
3 5	
4	
0 999999999 999999999 999999999	
1000000000 1000000000 1000000000 1000000000	
1 2	
1 3	
1 4	

提示

在第一个测试用例中，节点 1 没有子节点，因此对其执行操作不会改变其权值。故最大可能的总和为 3。

在第三个测试用例中，Vihaan 可以对节点 1 执行三次操作：

1. $[0, 2, 3] \rightarrow [2, 2, 3]$ 。
2. $[2, 2, 3] \rightarrow [4, 2, 3]$ 。
3. $[4, 2, 3] \rightarrow [6, 2, 3]$ 。

最终得到的总和为 $6 + 2 + 3 = 11$ 。可以证明不存在能获得更大总和的操作序列。

在第四个测试用例中，Vihaan 可以执行以下操作：

1. 对节点 2 执行操作：

$$[0, 0, 1] \rightarrow [0, 1, 1]$$

2. 对节点 1 执行四次操作：

$$[0, 1, 1] \rightarrow [1, 1, 1] \rightarrow [2, 1, 1] \rightarrow [3, 1, 1] \rightarrow [4, 1, 1]$$

最终得到的总和为 $4 + 1 + 1 = 6$ 。可以证明不存在能获得更大总和的操作序列。

Problem H. Deque Malfunction

时间限制: 3 秒

内存限制: 256 MB

输入文件: 标准输入流

输出文件: 标准输出流

题目描述

Vihaan 自己实现了一个双端队列 (deque)，但它有时会出现故障。

Vihaan 希望双端队列最终的内容从队头到队尾依次为 $[1, 2, \dots, n]$ 。

给定一个长度为 m 的数组 b 。元素将按照 $b_1, b_2, \dots, b_m$ 的顺序逐个到达。

每当一个元素 b_i 到达时，Vihaan 必须执行以下操作之一：

- 将 b_i 推入双端队列的队头。
- 将 b_i 推入双端队列的队尾。
- 使双端队列发生故障，并将 b_i 插入到双端队列中由他选择的任意位置。

Vihaan 还可以在任意时刻执行以下操作（可以执行任意多次）：

- 弹出双端队列队头的元素。
- 弹出双端队列队尾的元素。

在所有 m 个元素到达并完成所有操作后，双端队列必须恰好依序包含 $[1, 2, \dots, n]$ 。

求为了构造出 $[1, 2, \dots, n]$ ，双端队列发生故障的最少次数；如果无法达成目标，则输出 -1 。

输入格式

第一行包含一个整数 t ($1 \leq t \leq 10^4$) —— 测试用例的数量。

每个测试用例的第一行包含两个整数 n 和 m ($1 \leq n \leq m \leq 2 \cdot 10^5$) —— 目标双端队列的长度以及到达元素的数量。

每个测试用例的第二行包含 m 个整数 $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq n$) —— 元素到达的顺序。

保证所有测试用例的 m 之和不超过 $2 \cdot 10^5$ 。

输出格式

对于每个测试用例，输出一个整数——双端队列发生故障的最少次数。

如果无法得到目标双端队列，输出 -1 。

样例

标准输入流	标准输出流
9	0
1 3	-1
1 1 1	2
3 4	0
2 1 1 2	1
4 4	4
1 4 2 3	0
4 5	2
4 3 4 2 1	3
5 8	
1 3 2 5 4 3 1 2	
6 6	
1 6 5 4 3 2	
4 6	
1 3 2 3 4 3	
6 8	
2 6 3 5 2 1 4 2	
6 8	
1 6 2 5 1 5 3 4	

提示

在第一个测试用例中，Vihaan 可以执行以下操作：

1. 将第一个 1 推入队头： $\square \rightarrow [1]$ ，然后立即将其弹出： $[1] \rightarrow \square$ 。
2. 将第二个 1 推入队头： $\square \rightarrow [1]$ ，然后立即将其弹出： $[1] \rightarrow \square$ 。
3. 将最后一个 1 推入队尾： $\square \rightarrow [1]$ 。

因此，不需要发生任何故障。

在第二个测试用例中，数值 3 从未在 b 中出现，因此无法获得目标双端队列。

在第三个测试用例中，Vihaan 可以执行以下操作：

1. 将 1 推入队尾： $\square \rightarrow [1]$ 。
2. 将 4 推入队尾： $[1] \rightarrow [1, 4]$ 。
3. 使双端队列发生故障，并将 2 插入到 1 和 4 之间： $[1, 4] \rightarrow [1, 2, 4]$ 。
4. 使双端队列发生故障，并将 3 插入到 2 和 4 之间： $[1, 2, 4] \rightarrow [1, 2, 3, 4]$ 。

可以证明，不存在使用少于 2 次故障就能构造出目标双端队列的操作序列，因此最小值为 2。

在第四个测试用例中，Vihaan 可以执行以下操作：

1. 将第一个出现的 4 推入队头： $\square \rightarrow [4]$ ，然后立即将其弹出： $[4] \rightarrow \square$ 。
2. 将 3 推入队尾： $\square \rightarrow [3]$ 。
3. 将第二个出现的 4 推入队尾： $[3] \rightarrow [3, 4]$ 。

4. 将 2 推入队头: $[3, 4] \rightarrow [2, 3, 4]$ 。

5. 将 1 推入队头: $[2, 3, 4] \rightarrow [1, 2, 3, 4]$ 。

因此，不需要发生任何故障。